CLINICAN
Diseño documentado

Sistema de gestión de citas y fichas para la peluquería canina de la veterinaria CLINICAN, en Pasto.

Grupo	5B
Autor	Sebastian Lopez Benavides
Entregables	1 · Diseño documentado (este documento) 2 · Esqueleto funcional (repositorio `seba4s/Clinican`)
Versión	1.0 · noviembre de 2026

Contenido

1. El dominio y sus drivers
2. Árbol de utilidad y restricciones
3. Escenarios de calidad
4. Estilo arquitectónico elegido
5. Contextos delimitados
6. Tácticas aplicadas
7. Diagramas C4 (niveles 1 y 2) y vista de capas
8. Decisiones de arquitectura (ADR-001 a 003)
9. Riesgos identificados
10. Esqueleto funcional y trazabilidad
11. Cambios respecto a la propuesta

1 El dominio y sus drivers

CLINICAN atiende mascotas con cita previa. La recepción agenda los turnos, el personal de peluquería los atiende y registra en una ficha lo que se hizo, y la administradora define precios, cupos, horarios y personal. El dueño de la mascota no opera el sistema: acepta los términos al llegar y recibe recordatorios.

Lo que hace al dominio no trivial no son los formularios sino las reglas:

- **La capacidad de una franja depende de la mascota.** Cada franja de 30 minutos admite un número máximo de mascotas por *categoría de cupo*, y la categoría se calcula con la raza, el tamaño, el pelaje y el servicio: dos medianas a máquina caben juntas, pero un pastor alemán o un husky ocupan su propio cupo.
- **Un turno no es firme hasta que se paga el abono.** Nace pendiente, tiene 30 minutos para recibir $20.000 y, si no, se libera. Si varios pendientes compiten por el último cupo, gana el primero que paga y los demás se liberan.
- **Las inasistencias tienen consecuencias** que dependen de cuándo avisó el dueño (12 horas), y se registran en el historial.
- **No se atiende sin autorización:** términos aceptados y autorización de datos personales (Ley 1581 de 2012), con versión del texto y fecha.

Los tres drivers

Driver	Qué exige	De dónde sale
D1 · Disponibilidad sin conexión	La recepción agenda y consulta la agenda aunque no haya internet. Si la agenda no funciona, el negocio no atiende.	Condición real de operación: el local no tiene internet estable.
D2 · Integridad de la agenda	Nunca se sobrepasa la capacidad de una franja, ni siquiera cuando hay pendientes compitiendo o turnos que expiran mientras alguien registra un abono.	Es la regla que más afecta el servicio: una franja sobrevendida es un cliente esperando o una peluquera con dos mascotas a la vez.
D3 · Modificabilidad de las notificaciones	Agregar o cambiar el canal de recordatorios (hoy WhatsApp, mañana SMS o correo) no obliga a tocar la lógica de la agenda.	La API de WhatsApp de Meta cambia sus reglas, exige plantillas aprobadas y conexión para enviar.

2 Árbol de utilidad y restricciones

Cada hoja es un escenario concreto marcado con **(importancia para el negocio, dificultad técnica)** en Alta, Media o Baja. Las hojas resaltadas en verde son las que priorizamos: alta importancia y dificultad alta o media. Son cuatro de trece; el resto se atiende, pero no dirige la estructura.

Utilidad de CLINICAN

- **Disponibilidad**
  - Operación sin red
    - E1 · Con el internet caído, la recepción agenda un turno y ve la agenda del día en menos de 2 s. (A) (M)
  - Recuperación
    - E4 · Si el disco se daña, se restaura el último respaldo en menos de 10 minutos, perdiendo como máximo el día en curso. (A) (M)
    - Si falla el envío de un recordatorio, la agenda sigue funcionando y el envío se reintenta. (M) (B)
- **Integridad**
  - Capacidad de la franja
    - E2 · Dos turnos pendientes compiten por el último cupo: solo uno se confirma y 0 franjas quedan con más mascotas que su cupo. (A) (A)
  - Ciclo de vida
    - Un turno solo cambia de estado por una transición válida (por ejemplo, no se entrega lo que no se atendió). (M) (M)
    - Una actualización del programa migra la base de datos sin perder registros. (M) (M)
- **Modificabilidad**
  - Canales de notificación
    - E3 · Cambiar WhatsApp por SMS se hace agregando un adaptador, sin modificar agenda ni dominio, en menos de 1 día-persona. (A) (M)
  - Reglas configurables
    - La administradora cambia precios, cupos y horarios desde la pantalla, sin programar. (M) (B)
    - Un cambio en el texto legal crea una versión nueva y pide aceptarla de nuevo. (M) (B)
- **Seguridad y privacidad**
  - Ley 1581 de 2012
    - E5 · No se inicia la atención ni se envía un recordatorio a un dueño sin autorización vigente (100 % de los casos). (A) (B)
  - Control de acceso
    - Bloquear días, restaurar respaldos y cambiar precios solo lo hace la administradora, y queda en la auditoría. (M) (B)
- **Usabilidad**
  - Una persona nueva agenda un turno en menos de 2 minutos sin capacitación. (M) (M)
- **Rendimiento**
  - La agenda del día abre en menos de 1 s con cinco años de historial. (B) (B)

(A) alta (M) media (B) baja Primer chip: importancia · segundo chip: dificultad hoja priorizada

Restricciones (no se negocian)

Restricción	Tipo	Efecto en el diseño
Internet inestable en el local	Condición de operación	Nada que la agenda necesite puede depender de la red.

Un solo computador con Windows 11	Condición de operación	Sin servidor propio; la aplicación y los datos viven en el mismo equipo.
Ley 1581 de 2012 (datos personales)	Norma	Autorización con versión y fecha antes de atender o contactar; auditoría.
API oficial de WhatsApp (Meta)	Integración	Solo plantillas aprobadas fuera de la ventana de 24 h; requiere conexión y un token que no puede ir en el código.
Presupuesto mínimo; personal con poca experiencia en computadores	Negocio	Sin licencias ni hosting pagado; letra grande y flujos cortos.

3 Escenarios de calidad

Los tres primeros salen de las hojas priorizadas y son los que dirigen la estructura. E4 y E5 se incluyen porque restringen decisiones que tomamos por los primeros.

Parte	**E1 · Agendar sin internet** (D1)	**E2 · Último cupo en disputa** (D2)	**E3 · Cambiar de canal** (D3)
Fuente	Persona de recepción	Dos clientes y la recepción	Administradora del negocio con un desarrollador
Estímulo	Agenda un turno nuevo y consulta la agenda del día	Dos turnos pendientes piden el último cupo GRANDE de las 10:00; se registra el abono de uno mientras el temporizador de expiración revisa los vencidos	Decide enviar los recordatorios por SMS en vez de WhatsApp
Artefacto	Agenda: servicio de turnos y base de datos	Reglas de cupo y servicio de turnos	Módulo de notificaciones
Entorno	Operación normal, sin conexión a internet desde hace horas	Operación normal, franja con un cupo libre	Tiempo de desarrollo, sistema en producción
Respuesta	El turno queda guardado y se muestra; ninguna parte del flujo espera a la red	El primer abono confirma su turno; el otro pendiente pasa a LIBERADO con motivo «otro pagó primero» y se avisa al personal	Se escribe un adaptador nuevo que implementa el puerto `Notificador` y se conecta en el arranque (`main.py`)
Medida	100 % de los casos de uso de la agenda funcionan sin red; respuesta < 2 s	0 franjas con ocupación mayor que su cupo; verificado por pruebas automáticas que reproducen la carrera	0 líneas cambiadas en la agenda (servicio de turnos y reglas de cupo); solo el adaptador y su nombre en la lista de canales; < 1 día-persona

Parte	**E4 · Disco dañado** (disponibilidad)	**E5 · Atender sin autorización** (Ley 1581)
Fuente	Falla del hardware	Persona de peluquería
Estímulo	La base de datos queda ilegible	Intenta iniciar la atención de un cliente que no ha aceptado los términos ni la autorización de datos
Artefacto	Base de datos y respaldos	Servicio de turnos y servicio legal
Entorno	Fin de la jornada, con respaldo automático del día anterior	Llegada del cliente, sin conexión
Respuesta	La administradora restaura desde la pantalla Respaldo; antes, el sistema guarda una copia de lo actual y verifica la integridad del archivo	La acción se rechaza con un mensaje que lleva a la ficha, donde se registran los datos y la aceptación
Medida	Restauración < 10 min; pérdida ≤ 1 día de registros	0 atenciones iniciadas sin aceptación vigente

4 Estilo arquitectónico elegido

Monolito de escritorio en capas, con el dominio al centro y puertos y adaptadores en el borde de integración. Un solo proceso de Python con cuatro capas —presentación, aplicación, datos y dominio— y una base de datos SQLite embebida en el mismo equipo. Las dependencias apuntan en una sola dirección, hacia el dominio. La integración con WhatsApp entra por un puerto definido en la capa de aplicación y se conecta mediante un adaptador, con una bandeja de salida (*outbox*) que guarda los mensajes hasta que haya red.

Driver	Por qué este estilo lo atiende
D1 · Disponibilidad sin conexión	Todo lo que la agenda necesita está en el mismo computador: proceso, reglas y datos. No hay salto de red en ningún caso de uso de la agenda.
D2 · Integridad de la agenda	Un solo proceso y una sola base con transacciones ACID permiten volver a validar el cupo dentro de una transacción con bloqueo de escritura. No hay que coordinar réplicas.
D3 · Modificabilidad de las notificaciones	El puerto `Notificador` aísla el canal: la agenda produce «recordatorios pendientes» y no sabe quién los envía ni cómo.

Qué sacrifica

- **Escalabilidad y acceso remoto.** Funciona en un solo computador. Nadie puede ver la agenda desde el celular ni desde otra sede, y un segundo computador no puede compartir la base de forma segura (riesgo R3).
- **Inmediatez de los recordatorios.** Se envían cuando hay conexión, no en el instante en que se generan.
- **Despliegue independiente.** Cualquier cambio, aunque sea solo del canal, implica instalar una nueva versión del programa completo.
- **Un punto único de falla físico:** el disco del computador. Se mitiga con respaldos (táctica T4), no con redundancia.

Estilos que evaluamos y descartamos

Alternativa	Por qué no
Microservicios (agenda, fichas, notificaciones)	Agrega la red entre servicios justo donde D1 exige no depender de ella, y obliga a coordinar la integridad del cupo entre procesos (D2). Para un computador y cuatro usuarios, el costo de operación no se justifica.
Aplicación web en la nube	Si se cae internet, la agenda no abre: viola D1 de entrada.
Web local con sincronización a la nube (*offline-first*)	Resuelve el acceso remoto, pero la sincronización con conflictos es la peor enemiga de D2: dos copias podrían confirmar el mismo cupo. Es la opción para un crecimiento futuro (ver R3), no para hoy.
Eventos con un broker de mensajes	Desacopla las notificaciones (D3), pero exige operar un broker en un PC de recepción. La bandeja de salida en SQLite logra el mismo desacople sin otra pieza que instalar.

5 Contextos delimitados

Dentro de cada capa, el código se organiza por contexto: cada uno tiene su módulo en `dominio`, `datos` y `servicios`. Trazamos una frontera donde aparece una de estas señales: el mismo término cambia de significado, el ciclo de vida es distinto, o cambia por razones distintas o a ritmo distinto.

Contexto	Responsabilidad	Señal que justifica la frontera
Agenda	Franjas, cupos por categoría, turnos, abonos, expiración, grupos de mascotas, bloqueos	Ciclo de vida propio, con estados de minutos y horas (pendiente → confirmado → liberado). Aquí «servicio» es solo un *tipo* que define la categoría de cupo.
Atención (fichas)	Ficha de servicio, detalles del corte, precios, sesiones de desenredado	«Servicio» significa otra cosa: lo que realmente se hizo, con precio y observaciones. Su ciclo dura días (desenredado por sesiones) y cambia cuando cambian los precios, no la agenda.
Clientes	Propietarios, mascotas, razas, clientes sin registrar, importación de fichas antiguas	Datos de referencia que usan todos los demás y que viven años; se unen, corrigen e importan sin tocar turnos.
Legal	Textos versionados, consentimientos, declaración de responsabilidad	Cambia por una norma (Ley 1581), no por el negocio. Su unidad es la «versión aceptada con fecha», que nadie más maneja.
Notificaciones	Recordatorios, plantillas, bandeja de salida, canales	Cambia por un tercero (Meta) y a otro ritmo. Es la frontera que pide D3.
Acceso y operación	Personal, PIN, permisos, auditoría, configuración, respaldos	Transversal: ninguna regla de la agenda depende de cómo se autentica la gente o de cómo se respalda.

Cómo se relacionan

- **Agenda → Notificaciones:** la agenda no llama al canal. Escribe un hecho en la bandeja de salida («recordar el turno 812 mañana a las 09:00») dentro de la misma transacción que confirma el turno. Notificaciones lo lee y lo traduce a su propio modelo.
- **Agenda y Atención → Legal:** antes de iniciar la atención, o de contactar a un dueño, preguntan a Legal si hay autorización vigente.
- **Todos → Clientes:** referencian propietarios y mascotas por identificador.

6 Tácticas aplicadas

Táctica	Contra	Cómo se aplica en CLINICAN	Qué degrada
T1 · Eliminar la dependencia remota (disponibilidad)	E1	Base de datos embebida (SQLite) y reglas locales. La red solo la usa el despachador de notificaciones.	Acceso remoto y uso desde varios equipos.

T2 · Transacción con bloqueo de escritura y revalidación (integridad)	E2	Toda escritura de un caso de uso va dentro de `BEGIN IMMEDIATE`. Antes de confirmar un turno se vuelve a contar la ocupación de la franja dentro de esa misma transacción. Si ya no hay cupo, se rechaza y nada queda a medias.	Concurrencia: un solo escritor a la vez. Es irrelevante con un computador; sería un cuello de botella con muchos.
T3 · Reglas en un núcleo puro y máquina de estados (integridad)	E2	Categoría de cupo, cupos libres, precios y transiciones de estado son funciones del `dominio` sin acceso a la base, probadas aisladamente.	Algo de duplicación: la consulta trae datos y el dominio decide.
T4 · Respaldo verificado y recuperación al iniciar (disponibilidad)	E4	Respaldo diario y al cerrar, con la API de copia de SQLite y verificación `quick_check`; se conservan los últimos N. Al restaurar, primero se guarda lo actual. Al abrir, se liberan los turnos vencidos.	Espacio en disco; el cierre tarda un poco más.
T5 · Encapsular con un puerto (modificabilidad)	E3	El puerto `Notificador` se define en `servicios` y los adaptadores (WhatsApp de Meta, archivo o consola) viven en `adaptadores`.	Una indirección más: hay que leer dos archivos para seguir un envío.
T6 · Intermediario persistente (bandeja de salida) (modificabilidad y disponibilidad)	E1, E3	Los recordatorios se guardan en la tabla de salida en la misma transacción del turno. Un despachador, en un hilo aparte con su propia conexión, los envía cada minuto cuando hay red y reintenta los fallidos con espera creciente.	Latencia de entrega y una tabla que puede crecer si la red falta por días.
T7 · Restringir dependencias (modificabilidad)	E3	Una prueba automática falla si una capa importa a otra que no debe: por ejemplo, si `ui` importa `datos`, o si `dominio` importa cualquier otra capa.	Más fricción al programar: no se puede «tomar un atajo» aunque sea más rápido.
T8 · Autenticar, autorizar y auditar (seguridad)	E5	PIN con hash y sal, una tabla de permisos que solo consulta la capa de servicios, consentimientos versionados y registro de auditoría.	Usabilidad: hay que escribir el PIN y aceptar los términos en la ficha.

7 Diagramas C4

Nivel 1 · Contexto del sistema

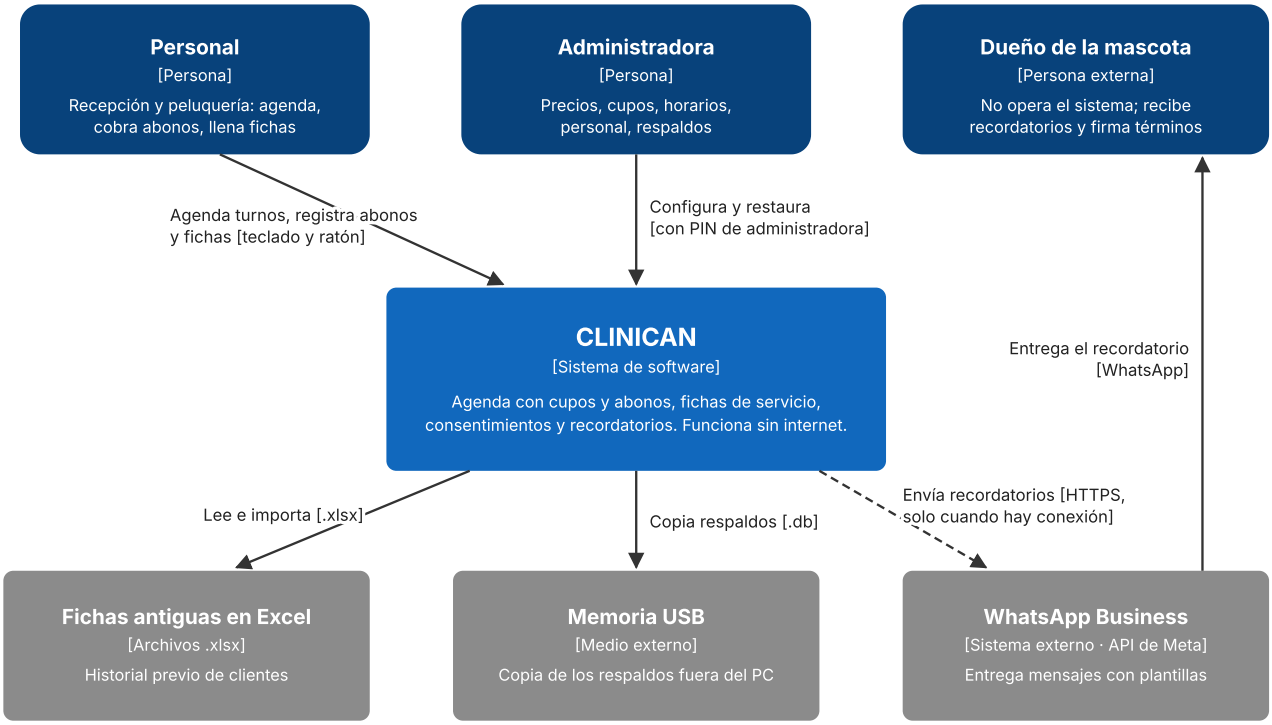


Figura 1. Contexto. Leyenda: azul oscuro = persona; azul = el sistema que diseñamos; gris = sistema o medio externo. Flecha continua = interacción síncrona y local; flecha discontinua = depende de internet y es asíncrona (no bloquea la agenda).

Nivel 2 · Contenedores

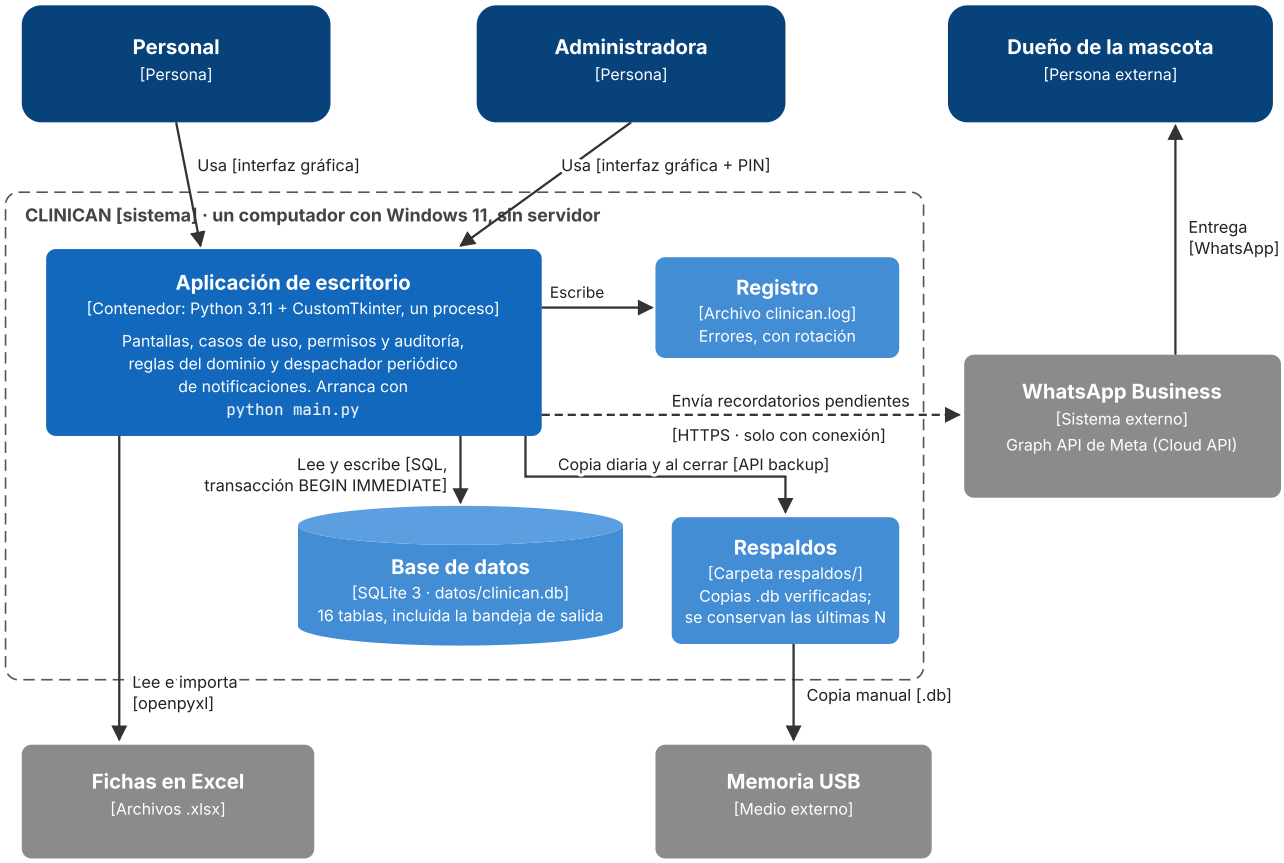


Figura 2. Contenedores. Leyenda: azul fuerte = contenedor que ejecuta código; azul claro = almacenamiento; gris = externo; borde discontinuo = frontera del sistema (un solo computador). Flecha discontinua = sale por internet de forma asíncrona. La aplicación y la base de datos comparten el equipo: ninguna flecha de la agenda cruza la red.

Vista complementaria · capas dentro de la aplicación

No la pide la entrega, pero es la que el esqueleto demuestra: cómo se reparten las responsabilidades dentro del contenedor «Aplicación de escritorio» y hacia dónde apuntan las dependencias.

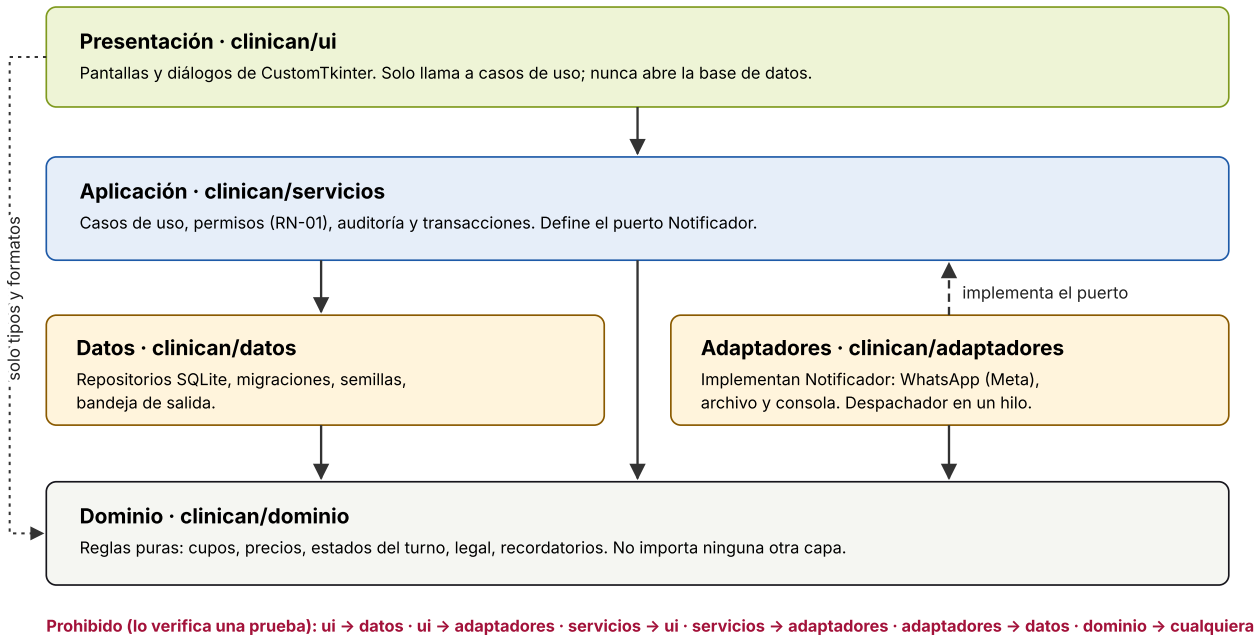


Figura 3. Capas. Flecha = «depende de» (importa). Todas apuntan hacia el dominio. La flecha punteada de la izquierda es la única dependencia que salta una capa: la presentación usa del dominio solo catálogos y formatos, nunca reglas con efectos. Los adaptadores se conectan en el arranque (`main.py` , la raíz de composición).

8 Decisiones de arquitectura

Cada ADR tiene cinco secciones: estado, contexto, decisión, alternativas descartadas y consecuencias, buenas y malas.

ADR-001 · Monolito de escritorio en capas con base de datos local

ESTADO

Aceptada.

CONTEXTO

D1 exige que la agenda funcione sin internet, y la restricción de operación es un solo computador con Windows, sin presupuesto para un servidor. Lo usan cuatro personas, siempre en el mismo mostrador. D2 pide una integridad fuerte del cupo, más fácil de garantizar con una sola base de datos.

DECISIÓN

Aplicación de escritorio en Python con cuatro capas (ui, servicios, datos, dominio) en un solo proceso, y SQLite como base de datos embebida en `datos/clinican.db`. La regla de dependencias es una sola dirección hacia el dominio, y la verifica una prueba automática.

ALTERNATIVAS DESCARTADAS

- **Aplicación web en la nube** (por ejemplo, Django en un hosting). Sin internet no hay agenda: viola D1.
- **Microservicios** para agenda, fichas y notificaciones. Mete la red entre las piezas, obliga a coordinar el cupo entre procesos y a operar varios servicios en un PC de recepción. Es más costo que beneficio para este tamaño.
- **Servidor local con cliente web** (Flask en el mismo PC más navegador). Cumple D1, pero agrega un servidor que levantar y vigilar, sin una ganancia hoy, porque nadie más entra a la agenda.

CONSECUENCIAS

- Buena: la agenda nunca depende de la red; instalación con un solo ejecutable (PyInstaller).
- Buena: las transacciones ACID locales hacen posible ADR-002 sin coordinación distribuida.
- Mala: no hay acceso remoto ni uso desde varios computadores. Abrir la misma base desde dos equipos por la red corrompe datos (R3).
- Mala: el disco del PC es un punto único de falla; dependemos de los respaldos (T4, R1).
- Mala: si el negocio crece, migrar a cliente-servidor significa reescribir la presentación (los servicios y el dominio se conservan).

ADR-002 · Agenda por franjas con cupos por categoría, protegida por una transacción con bloqueo

ESTADO

Aceptada. Reemplaza el modelo de la propuesta: agenda por peluquero con duración variable.

CONTEXTO

D2: nunca sobrevender. Al levantar los requisitos con el negocio encontramos que no asigna peluquero al agendar y que trabaja con franjas fijas de 30 minutos. Lo que varía es cuántas mascotas caben por franja, según la categoría: MÁQUINA 2, GRANDE 1, TIJERA 1, COMPLICADO 1. Además, hay carreras reales: un turno pendiente puede expirar justo cuando se registra su abono, y varios pendientes compiten por el último cupo.

DECISIÓN

- La categoría y los cupos libres se calculan en el dominio con funciones puras.
- Cada caso de uso que ocupa un cupo (crear, abonar, mover) abre `BEGIN IMMEDIATE`, vuelve a contar la ocupación de la franja dentro de la transacción y confirma solo si cabe.
- Cuando un abono agota el cupo, en esa misma transacción los demás pendientes pasan a LIBERADO («otro pagó primero»).
- Los estados del turno siguen una máquina de estados del dominio.

ALTERNATIVAS DESCARTADAS

- **Agenda por peluquero con duración variable** (lo que propusimos al inicio). No refleja cómo trabaja el negocio y obligaría a estimar la duración de algo que depende de nudos y conducta. Se descartó tras hablar con la administradora.
- **Validar el cupo solo en la pantalla.** Entre ver el cupo libre y guardar puede pasar el temporizador de expiración o un segundo abono; la validación tiene que estar donde se escribe.
- **Restricción o trigger en SQL.** Una restricción no expresa «como máximo N por categoría según la configuración», y un trigger escondería la regla de negocio en la base, lejos de las pruebas del dominio.
- **Bloqueo optimista con número de versión.** Sirve con muchos escritores; aquí hay uno solo y obligaría a reintentar en la interfaz sin ganancia.

CONSECUENCIAS

- Buena: la regla vive en un solo lugar del dominio, con pruebas que reproducen las carreras.
- Buena: los cupos se cambian en Configuración, sin programar.
- Mala: un solo escritor a la vez. Con un PC no se nota; con varios equipos sería el primer cuello de botella.
- Mala: no se sabe qué peluquera atiende cada turno hasta que se llena la ficha. Si el negocio lo quiere, hay que extender el modelo.

ADR-003 · Recordatorios por un puerto y una bandeja de salida en la base local

ESTADO

Aceptada.

CONTEXTO

D3 exige cambiar el canal sin tocar la agenda, y D1 exige que la agenda no espere a la red. La API de Meta pide plantillas aprobadas, conexión y un token. La Ley 1581 exige autorización antes de contactar al dueño.

DECISIÓN

- Al confirmar o mover un turno, el servicio de turnos guarda un recordatorio en la tabla de salida, en la misma transacción.
- Un despachador, en un hilo aparte con su propia conexión, lee los pendientes cada minuto y los entrega a un `Notificador`, un puerto definido en `servicios`. Justo antes de enviar, el dominio revisa si todavía tiene sentido: el turno sigue confirmado, no ha empezado y el dueño tiene autorización de datos vigente (Ley 1581). Si no, lo descarta con su motivo. Si el canal falla, reintenta con espera creciente.
- Los adaptadores viven en `adaptadores`: `NotificadorWhatsApp` (HTTPS a la Graph API, con plantilla aprobada) y `NotificadorArchivo` (escribe el mensaje en un archivo, para operar sin cuenta de Meta). `main.py` elige uno según el canal de la configuración y se lo pasa al despachador; las capas internas no importan ningún adaptador.

ALTERNATIVAS DESCARTADAS

- **Llamar a la API de WhatsApp desde el servicio de turnos.** Es lo más rápido de escribir, pero acopla la agenda al canal (viola D3) y la bloquea si no hay red (viola D1).
- **Un microservicio de notificaciones en la nube.** Aísla bien el canal, pero exige sincronizar la agenda hacia la nube y operar un servidor pagado.
- **Un broker de mensajes** (RabbitMQ o similar) entre la agenda y el canal. Da el mismo desacople que la bandeja, con una pieza más que instalar y mantener en el PC de recepción.

CONSECUENCIAS

- Buena: un canal nuevo es un adaptador nuevo y su nombre en la lista de canales; la agenda y sus reglas no cambian (E3).
- Buena: sin red, los recordatorios se acumulan y salen cuando vuelve la conexión; nada se pierde porque está en la misma base y en los mismos respaldos.
- Mala: el recordatorio puede llegar tarde si la red falta mucho tiempo. Uno vencido no se envía y queda marcado.
- Mala: entrega «al menos una vez»: si se cae justo después de enviar y antes de marcar, el dueño puede recibir el mensaje dos veces.
- Mala: el envío real depende de una cuenta de Meta verificada y de plantillas aprobadas, que no controlamos (R2).

9 Riesgos identificados

#	Riesgo	Impacto	Qué habría que verificar para descartarlo
R1	Los respaldos quedan en el mismo disco del computador. Un robo, un ransomware o un disco dañado se lleva la base y las copias.	Pérdida total del historial y de los consentimientos (Ley 1581). Afecta D1 y E4.	Hacer un simulacro: copiar a la USB, borrar la base en un equipo de prueba, restaurar y medir el tiempo y lo perdido. Confirmar con el negocio quién y cada cuánto saca la copia del equipo.
R2	Meta no aprueba la plantilla, cambia la política del número o el costo por conversación hace inviable el canal.	No salen recordatorios. La agenda sigue funcionando gracias a ADR-003, pero el valor de D3 se pierde hasta tener otro canal.	Crear la cuenta de Meta Business, someter la plantilla de recordatorio, enviar un mensaje real de prueba y consultar la tarifa vigente para Colombia. Revisar qué pasa con el uso normal del número 301 441 7194 al pasar a la API.
R3	El negocio crece y pone un segundo computador abriendo el mismo archivo SQLite por la red local.	SQLite no está hecho para compartirse por carpetas de red: bloqueos y posible corrupción. Es lo primero que se rompe si el negocio crece diez veces.	Probar dos equipos escribiendo a la vez sobre una carpeta compartida de Windows y medir errores. Mientras tanto, la instrucción es un solo equipo. La salida sería un servidor local o en la nube detrás de la misma capa de servicios.
R4	La bandeja de salida crece sin límite si la red falta por semanas o el token vence.	Recordatorios inútiles enviados tarde y la base creciendo.	Ejecutar el despachador con la red cortada por 30 días simulados y comprobar que los vencidos se descartan y que la tabla se purga.

Si el negocio creciera diez veces

Diez veces más turnos en el mismo local no rompen nada: SQLite maneja millones de filas y la agenda se consulta por fecha. Diez veces significa en realidad varias sedes o varios computadores. Lo primero que se rompe es **ADR-001**, el monolito en un solo equipo (R3), y con él la táctica de un solo escritor de ADR-002. Lo que sobrevive es el dominio y los servicios: la frontera de capas permite poner un servidor detrás de la misma capa de servicios sin reescribir las reglas.

10 Esqueleto funcional y trazabilidad

El esqueleto es el mismo repositorio del programa. Para la sustentación, un comando recorre el sistema de punta a punta sin abrir ventanas y muestra cada capa en acción:

```
python -m clinican.demo
```

1. **Estructura:** lee el código fuente y muestra de qué capa depende cada una, y que no hay atajos (ADR-001).
2. **D2:** con una base temporal, dos labradores piden el último cupo GRANDE de las 10:00. Gana quien paga primero, el otro se libera y un abono tardío se rechaza (ADR-002).
3. **D1:** el despachador intenta enviar el recordatorio sin internet; queda pendiente con su error, y mientras tanto se agenda otro turno sin esperar (ADR-003).
4. **D3:** vuelve la red y el recordatorio sale por archivo; luego el mismo mensaje sale por un canal SMS escrito dentro de la demo y por consola, sin tocar la agenda. El recordatorio

de un dueño sin autorización de datos se descarta (Ley 1581).

La regla de capas se comprueba con `python -m pytest tests/test_arquitectura.py`: analiza los *imports* de cada archivo y además verifica que solo la capa de datos ejecute SQL. Incluye casos que plantan un atajo a propósito para demostrar que la prueba lo detecta. El programa completo, con ventanas, se abre con `python main.py`. Las instrucciones están en `INSTRUCCIONES.md`, y el proyecto tiene 323 pruebas automáticas.

Driver	Decisión	Tácticas	Dónde está en el código	Cómo se comprueba
D1	ADR-001, ADR-003	T1, T4, T6	`datos/conexion.py`, `datos/archivo_bd.py`, `servicios/respaldos.py`, `datos/repo_notificaciones.py`	`tests/test_notificaciones.py`, `tests/test_respaldos.py`; parte 3 de la demo
D2	ADR-002	T2, T3	`dominio/cupos.py`, `dominio/turnos.py`, `servicios/turnos.py`	`tests/test_turnos.py`; parte 2 de la demo
D3	ADR-003	T5, T6, T7	Puerto en `servicios/notificaciones.py`; `adaptadores/`; `dominio/notificaciones.py`; raíz de composición en `main.py`	`tests/test_notificaciones.py`, `tests/test_arquitectura.py`; partes 1 y 4 de la demo

11 Cambios respecto a la propuesta

La propuesta se escribió antes de levantar los requisitos con el negocio. Mantuvimos los tres drivers y ajustamos el modelo donde la realidad era distinta:

Decía la propuesta	Quedó así	Por qué
Agenda por peluquero, con duración variable por raza y servicio	Franjas de 30 minutos con cupos por categoría, que se calculan con la raza, el tamaño, el pelaje y el servicio	El negocio no asigna peluquera al agendar; la variación por raza y tamaño se expresa en cuántas mascotas caben por franja (ADR-002). El driver D2 no cambia: nunca sobrevender.
Cinco tipos de usuario, incluidos el peluquero y el veterinario por separado	Dos roles que operan (Administradora y Personal) y el dueño como actor externo	Recepción y peluquería las hace el mismo personal. Los permisos cambian solo entre administradora y el resto.
El veterinario autoriza a mascotas sin vacunas	El requisito de carnet de vacunas es una cláusula de los términos que el dueño acepta al llegar	La revisión se hace con el carnet físico; no hay un flujo de aprobación en el sistema. Queda fuera del alcance de esta versión.